

RV-Sparse Coding Challenge

Zero-Allocation Sparse Matrix-Vector Multiplication

Abdelrhman Atta

May 12, 2026

1 Introduction

This document outlines the implementation details and mathematical formulation for the **RV-Sparse** coding challenge. The primary objective is to implement a highly efficient Sparse Matrix-Vector (SpMV) multiplication routine. The algorithm converts a dense, row-major matrix into the Compressed Sparse Row (CSR) format on-the-fly and computes the matrix-vector product under a strict *zero dynamic memory allocation* constraint.

2 Mathematical Formulation

2.1 Standard Matrix-Vector Multiplication

Let $A \in \mathbb{R}^{m \times n}$ be a dense matrix with m rows and n columns. Let $x \in \mathbb{R}^n$ be a dense column vector. The matrix-vector product $y = Ax$, where $y \in \mathbb{R}^m$, is defined element-wise as:

$$y_i = \sum_{j=0}^{n-1} A_{i,j} x_j \quad \text{for } i = 0, 1, \dots, m-1 \quad (1)$$

2.2 The Compressed Sparse Row (CSR) Representation

For a sparse matrix A , the majority of elements are zero. We define a zero-tolerance threshold $\epsilon = 10^{-12}$. An element is considered non-zero if $|A_{i,j}| > \epsilon$.

Let N_{nz} represent the total number of non-zero elements. The CSR format compresses A into three 1-dimensional arrays:

- $V \in \mathbb{R}^{N_{nz}}$ (**values**): Stores the non-zero values.
- $C \in \mathbb{N}^{N_{nz}}$ (**col_indices**): Stores the column index j corresponding to each value in V .
- $R \in \mathbb{N}^{m+1}$ (**row_ptrs**): Stores the starting index in V and C for each row i .

2.3 Example 1: Complex Matrix to CSR Mapping

Consider a sparse 5×5 matrix A with $m = 5$ rows. Note that Row 1 is completely empty, which is a common occurrence in sparse matrices:

$$A = \begin{bmatrix} 0.0 & 9.0 & 0.0 & -3.0 & 0.0 \\ 0.0 & 0.0 & 0.0 & 0.0 & 0.0 \\ 5.0 & 0.0 & 0.0 & 0.0 & 2.0 \\ 0.0 & 0.0 & 4.0 & 0.0 & 0.0 \\ 0.0 & 1.0 & 0.0 & -7.0 & 0.0 \end{bmatrix} \quad (2)$$

Applying the CSR compression protocol, we extract the $N_{nz} = 7$ non-zero elements:

- **Values (V):** $[9.0, -3.0, 5.0, 2.0, 4.0, 1.0, -7.0]$
- **Column Indices (C):** $[1, 3, 0, 4, 2, 1, 3]$
- **Row Pointers (R):** $[0, 2, 2, 4, 5, 7]$

Note on Row Pointers: $R_1 = 2$ and $R_2 = 2$. Because $R_1 = R_2$, the interval $[2, 2)$ is empty, correctly indicating that Row 1 contains zero non-zero elements. R_5 stores the total $N_{nz} = 7$.

2.4 Example 2: SpMV Execution Walkthrough

Using the CSR representation, the dense matrix-vector multiplication is optimized by entirely skipping zero-value computations. The formulation becomes:

$$y_i = \sum_{k=R_i}^{R_{i+1}-1} V_k \cdot x_{C_k} \quad \text{for } i = 0, 1, \dots, m-1 \quad (3)$$

Let us multiply the matrix A from Example 1 by a dense vector x :

$$x = \begin{bmatrix} 1.0 \\ 2.0 \\ 3.0 \\ 4.0 \\ 5.0 \end{bmatrix} \quad (4)$$

The algorithm processes the multiplication row by row:

- **Row 0 ($i = 0$):** Loops from $k = R_0 = 0$ to $R_1 - 1 = 1$.

$$\begin{aligned} y_0 &= V_0 \cdot x_{C_0} + V_1 \cdot x_{C_1} \\ &= (9.0 \cdot x_1) + (-3.0 \cdot x_3) \\ &= (9.0 \cdot 2.0) + (-3.0 \cdot 4.0) = 18.0 - 12.0 = \mathbf{6.0} \end{aligned}$$

- **Row 1 ($i = 1$):** Loops from $k = R_1 = 2$ to $R_2 - 1 = 1$. The loop condition fails ($k < 2$), meaning the sum remains initialized at zero. $y_1 = \mathbf{0.0}$.

- **Row 2** ($i = 2$): Loops from $k = R_2 = 2$ to $R_3 - 1 = 3$.

$$\begin{aligned} y_2 &= V_2 \cdot x_{C_2} + V_3 \cdot x_{C_3} \\ &= (5.0 \cdot x_0) + (2.0 \cdot x_4) \\ &= (5.0 \cdot 1.0) + (2.0 \cdot 5.0) = 5.0 + 10.0 = \mathbf{15.0} \end{aligned}$$

- **Row 3** ($i = 3$): Loops from $k = R_3 = 4$ to $R_4 - 1 = 4$.

$$y_3 = V_4 \cdot x_{C_4} = (4.0 \cdot x_2) = (4.0 \cdot 3.0) = \mathbf{12.0}$$

- **Row 4** ($i = 4$): Loops from $k = R_4 = 5$ to $R_5 - 1 = 6$.

$$\begin{aligned} y_4 &= V_5 \cdot x_{C_5} + V_6 \cdot x_{C_6} \\ &= (1.0 \cdot x_1) + (-7.0 \cdot x_3) \\ &= (1.0 \cdot 2.0) + (-7.0 \cdot 4.0) = 2.0 - 28.0 = \mathbf{-26.0} \end{aligned}$$

The resulting vector is $y = [6.0, 0.0, 15.0, 12.0, -26.0]^\top$. This logic drastically reduces computational complexity from $\mathcal{O}(m \times n)$ to $\mathcal{O}(m + N_{nz})$.

3 Algorithm and Memory Constraints

The implemented `sparse_multiply` function operates in two phases within a single subroutine:

1. **Compression Phase** ($\mathcal{O}(m \times n)$): The dense matrix A is scanned row by row. Non-zero elements are appended to V and C , while R is populated sequentially.
2. **Multiplication Phase** ($\mathcal{O}(m + N_{nz})$): Equation 3 is evaluated to populate the dense output vector y .

Zero Dynamic Memory Allocation: A critical constraint of this implementation is the strict avoidance of `malloc`, `calloc`, or `realloc`. All arrays (V , C , R , y) are provided by the caller as pre-allocated buffers. The function operates exclusively on these external pointers in $\mathcal{O}(1)$ auxiliary space.

4 Repository Information

The complete C source code, including the test harness and implementation logic, is hosted publicly on GitHub.

Git Repository: <https://github.com/AbdelrhmanAtta/rv-sparse>